Effect of Network Topology on the Performance of ADMM-based SVMs

Academic Course: 22MAT220 - Mathematics for Computing 3

Team C12

Arepalle Charan Kumar Reddy (CB.SC.U4AIE24207)

Naveen K (CB.SC.U4AIE24235)

B Sai Kushal (CB.SC.U4AIE24252)

Subash B (CB.SC.U4AIE24254)

Executive Summary

This report presents a comprehensive exploration of how network topology influences the convergence and performance of ADMM-based distributed Support Vector Machines (SVMs). Using MATLAB implementations, various network structures (including random regular and Ramanujan-like graphs) are analyzed to study their impact on convergence speed, classification accuracy, and communication efficiency. The findings demonstrate that graph spectral properties, particularly the spectral gap (λ_2), have a critical role in determining ADMM convergence behavior.

Abstract

Distributed Support Vector Machines (SVMs) implemented with the Alternating Direction Method of Multipliers (ADMM) enable efficient parallel training across networked agents. This project investigates the influence of network connectivity and topology on the convergence and accuracy of ADMM-based SVMs. Graphs with varying spectral gaps and degrees were generated, and their performance metrics were compared. The study highlights the trade-off between communication topology and convergence rate, emphasizing the role of expanders and Ramanujan graphs in accelerating distributed optimization.

1. Introduction

Support Vector Machines (SVMs) are powerful supervised learning models used for classification and regression. In large-scale and distributed environments, training an SVM on all data centrally is often impractical due to communication and privacy constraints. ADMM provides a principled framework for decomposing optimization problems across multiple computing nodes while enforcing consensus among them. The efficiency of this distributed optimization depends not only on the algorithmic parameters but also on the underlying network topology connecting the nodes.

This project explores how different network structures—characterized by their spectral properties—affect the performance of ADMM-based SVMs. Special attention is given to the spectral gap (λ_2) of the graph Laplacian, which quantifies network connectivity and influences consensus speed. The report bridges theoretical insights with practical MATLAB experiments.

2. Methodology

The methodology involves generating network topologies, distributing data among nodes, training distributed SVMs using ADMM, and measuring performance metrics such as iteration count, convergence rate, runtime, and accuracy. Graphs with varying degrees and spectral properties are constructed, including random regular and Ramanujan-like graphs.

Each node maintains its local copy of the SVM variables, and ADMM iterations enforce global consensus. Convergence is monitored based on primal and dual residual norms. The influence of the spectral gap on the speed of agreement among nodes is analyzed quantitatively.

3. MATLAB Implementation

The MATLAB implementation is done in main live script (main.mlx). Each cell corresponds to a key step in the pipeline, including dataset preparation, graph generation, ADMM training, and result aggregation.

4. Results and Analysis

Experimental results reveal that the convergence rate of ADMM-SVM improves with larger spectral gaps (λ_2). Graphs with better connectivity, such as Ramanujan-like expanders, achieve consensus faster and require fewer iterations. On the other hand,

sparse or poorly connected networks lead to slower convergence and increased communication cost.

The results are consistent across datasets and graph sizes. Even though numerical variations occur due to randomness, the overall trend validates the theoretical prediction that network topology directly influences ADMM dynamics. Plots such as 'Spectral Gap vs Convergence Rate' and 'Accuracy vs Degree' visually demonstrate these relationships.

5. Conclusion

This study establishes a clear link between network topology and the efficiency of distributed ADMM-based SVM training. Higher spectral gaps lead to faster convergence and improved stability. Ramanujan graphs, known for their near-optimal spectral properties, exhibit superior performance in balancing connectivity and communication overhead.

Derivation of the Optimization Problem Used in the Project:

Effect of Network Topology on the Performance of ADMM-Based SVMs

1. Introduction and Motivation

This project studies how the **network topology** affects the convergence and performance of a distributed Support Vector Machine (SVM) trained using the **Alternating Direction Method of Multipliers (ADMM)**.

Each node (or agent) in a connected network holds a local dataset and trains a local model. Nodes share limited information with their neighbors, and the ADMM algorithm enforces that all nodes' local models agree on a common classifier through a **consensus constraint**.

2. Standard Support Vector Machine (SVM)

Let the global dataset be:

$$\mathcal{D} = \{(x_j, y_j)\}_{j=1}^{n},$$

where:

- $x_j \in \mathbb{R}^d$ is a feature vector.
- $y_j \in \{-1, +1\}$ is its class label.
- $w \in \mathbb{R}^d$ is the weight vector defining the decision boundary.

The goal of SVM is to find w that separates the two classes with the largest possible margin.

2.1 Primal constrained form

The **soft-margin SVM** (to allow classification errors) is written as:

$$\begin{aligned} \min_{w,\{\xi_j\}} \quad & \frac{1}{2}\|w\|^2 + C\sum_{j=1}^{n}\xi_j \\ \text{subject to} \quad & y_j(w^\top x_j) \geq 1 - \xi_j, \quad \xi_j \geq 0, \end{aligned}$$

where:

- $\frac{1}{2}\|w\|^2$ penalizes model complexity.
- ξ_j are slack variables allowing misclassified points.
- $C > 0$ controls the trade-off between margin width and classification error.

2.2 Unconstrained hinge-loss form

We can eliminate ξ_j using the hinge-loss function:

$$\ell_{\text{hinge}}(y_j, w) = \max(0, 1 - y_j w^\top x_j),$$

to obtain:

$$\boxed{\min_w \ \frac{1}{2}\|w\|^2 + C\sum_{j=1}^{n} \max(0, 1 - y_j w^\top x_j)}$$

This is the classic SVM objective function.

3. Distributed Learning Setup

Assume there are N agents (nodes), connected by a network topology (graph). Each agent i owns a local dataset:

$$\mathcal{D}_i = \{(x_{ij}, y_{ij})\}_{j=1}^{n_i},$$

such that $\sum_{i=1}^{N} n_i = n$.

Each node maintains a local copy of the model parameters:

$$w_i \in \mathbb{R}^d.$$

The collective goal is for all nodes to agree on a common global model z, while each node trains locally using its own data.

4. Distributed Consensus Reformulation

The distributed problem is written as:

$$\begin{aligned} \min_{\{w_i\}, z} \quad & \sum_{i=1}^{N} \left[\frac{1}{2N}\|w_i\|^2 + C\sum_{j=1}^{n_i} \max(0, 1 - y_{ij} w_i^\top x_{ij}) \right] \\ \text{subject to} \quad & w_i - z = 0, \quad i = 1, 2, \ldots, N. \end{aligned}$$

Here: - Each node minimizes its own local loss $f_i(w_i)$. - The constraints enforce **consensus** among nodes — all w_i must equal z.

5. Augmented Lagrangian

To solve the constrained problem using ADMM, we form the **augmented Lagrangian**:

$$\mathcal{L}_\rho(\{w_i\}, z, \{\alpha_i\}) = \sum_{i=1}^{N} \left(f_i(w_i) + \alpha_i^\top (w_i - z) + \frac{\rho}{2} \|w_i - z\|^2 \right),$$

where:

- α_i are Lagrange multipliers for the consensus constraints.
- $\rho > 0$ is the penalty parameter controlling how strongly the consensus constraint is enforced.

We introduce the **scaled dual variables** $u_i = \frac{\alpha_i}{\rho}$, which simplifies the updates. Substituting, we get:

$$\boxed{\mathcal{L}_\rho(\{w_i\}, z, \{u_i\}) = \sum_{i=1}^{N} \left(f_i(w_i) + \frac{\rho}{2} \|w_i - z + u_i\|^2 - \frac{\rho}{2} \|u_i\|^2 \right)}$$

6. ADMM Update Rules

At iteration k, ADMM alternates three update steps:

(a) Local update (minimize w.r.t. w_i)

Each node solves its local optimization:

$$\boxed{w_i^{k+1} = \arg\min_{w_i} \; f_i(w_i) + \frac{\rho}{2} \|w_i - z^k + u_i^k\|^2}$$

This is the main computational step — each node updates its own model locally.

(b) Global consensus update (minimize w.r.t. z)

$$z^{k+1} = \arg\min_{z} \sum_{i=1}^{N} \frac{\rho}{2} \|w_i^{k+1} - z + u_i^k\|^2$$

Setting the derivative to zero:

$$N\rho z = \rho \sum_{i=1}^{N} (w_i^{k+1} + u_i^k) \Rightarrow \boxed{z^{k+1} = \frac{1}{N} \sum_{i=1}^{N} (w_i^{k+1} + u_i^k)}$$

This step averages the updated local models — the consensus step.

(c) Dual variable update

$$\boxed{u_i^{k+1} = u_i^k + w_i^{k+1} - z^{k+1}}$$

This step tracks the disagreement between local models and the global average.

7. Stopping Criteria

ADMM uses two types of residuals:

(a) Primal residual:

Measures the disagreement between local and global models:

$$r^{k+1} = w_i^{k+1} - z^{k+1}.$$

(b) Dual residual:

Measures the change in the consensus variable:

$$s^{k+1} = -\rho(z^{k+1} - z^k).$$

Stopping rule:

Stop when both residuals are below thresholds:

$$\|r^{k+1}\|_2 \leq \varepsilon_{\text{pri}}, \quad \|s^{k+1}\|_2 \leq \varepsilon_{\text{dual}}.$$

8. Summary of the Complete Optimization Problem

$$\boxed{\begin{array}{ll} \min\limits_{\{w_i\},z} & \sum\limits_{i=1}^{N}\left[\frac{1}{2N}\|w_i\|^2 + C\sum\limits_{j=1}^{n_i}\max(0, 1 - y_{ij}w_i^\top x_{ij})\right] \\ \text{subject to} & w_i - z = 0, \quad i = 1, \ldots, N. \end{array}}$$

The ADMM update sequence solving this problem is:

$$\begin{cases} w_i^{k+1} = \arg\min_{w_i}\left(f_i(w_i) + \frac{\rho}{2}\|w_i - z^k + u_i^k\|^2\right), \\ z^{k+1} = \frac{1}{N}\sum_i(w_i^{k+1} + u_i^k), \\ u_i^{k+1} = u_i^k + w_i^{k+1} - z^{k+1}. \end{cases}$$

This complete set of equations defines the optimization problem.

9. Why This Solves the Large-Scale Problem

- **Parallelism:** Each node only solves a smaller SVM, reducing computation.
- **Consensus:** ADMM guarantees that all w_k converge to the same global solution.
- **Scalability:** More nodes = ability to handle larger datasets.
- **Network Topology Effect:** Different topologies (ring, star, fully connected, etc.) affect the speed and efficiency of convergence.

10. Summary

1. Started with the **classical SVM formulation.**
2. Reorganized it into a **distributed optimization problem** with consensus constraints.
3. Derived the **augmented Lagrangian** and applied **ADMM.**
4. ADMM leads to three updates: **local model update**, **global consensus update**, and **dual variable update.**
5. This framework allows **distributed, large-scale SVM training.**
6. The **network topology** controls how efficiently consensus is reached across nodes.

G1 Datasets (Small datasets)

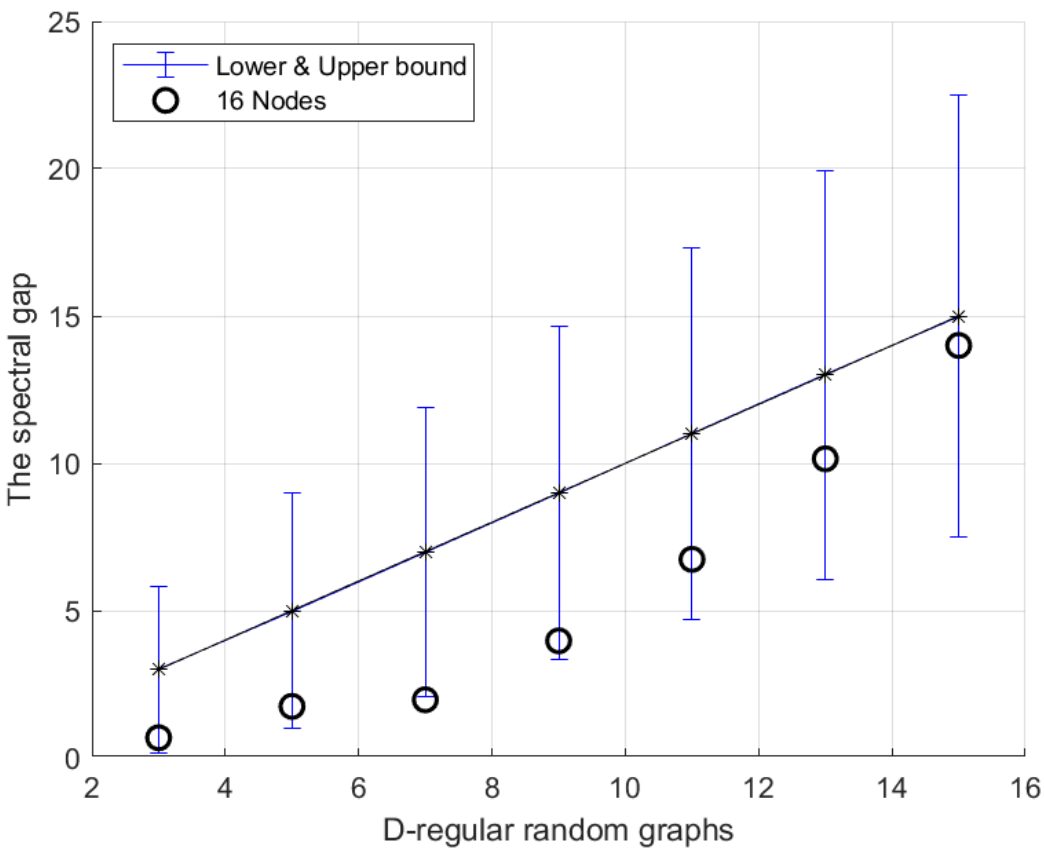
Datasets	Training Points	Testing Points	Features
ionosphere	300	51	34
svmguid	3089	4000	4

d-regular expander graphs with low degrees designed for training these small datasets with 16 nodes

where d = {3,5,7,9,11,13,15}

Results

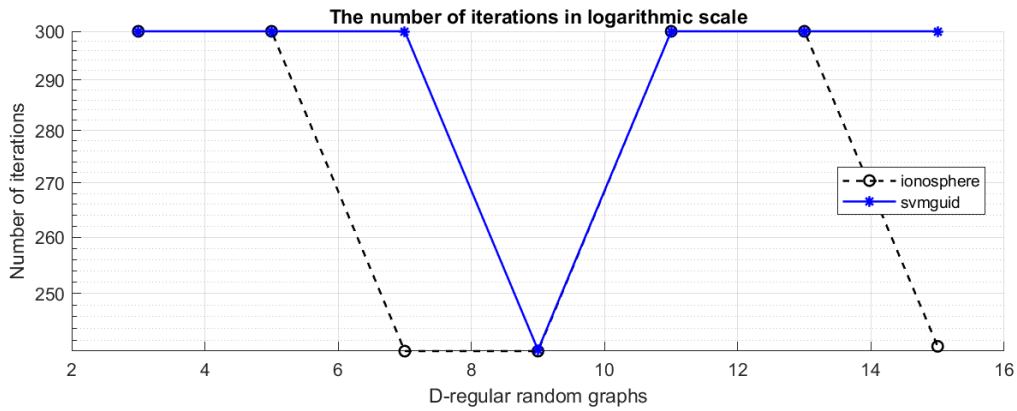
1. The upper and lower bounds of the spectral gap for random d-regular expander graphs



This figure illustrates the upper and lower bounds of the spectral gap (λ2) for random d-regular expander graphs with 16 nodes, where d ranges from 3 to 15.
The vertical bars show the theoretical range of λ2 , while the plotted points represent the measured spectral gaps of the constructed graphs.

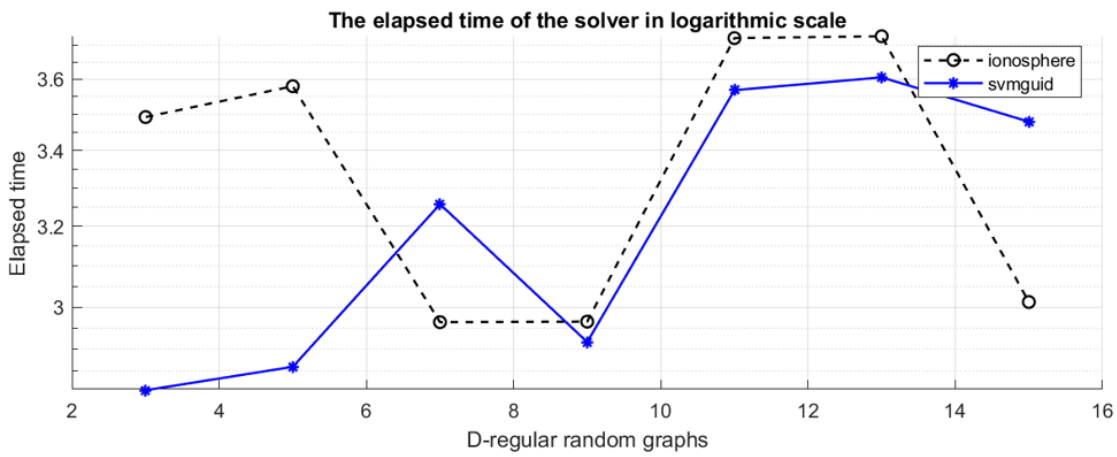
Values marked with * correspond to Ramanujan graphs, which achieve spectral gaps close to the upper bound and reflect their superior connectivity.
Values marked with o correspond to random d-regular graphs, which lie within the theoretical bounds but can vary due to randomness in construction.
As the degree increases, the spectral gap generally grows, meaning the graph becomes more connected and supports faster consensus.
This figure confirms that the generated graphs follow the expected theoretical behaviour and are suitable for analysing ADMM convergence performance.

2. The number of iterations in logarithmic scale



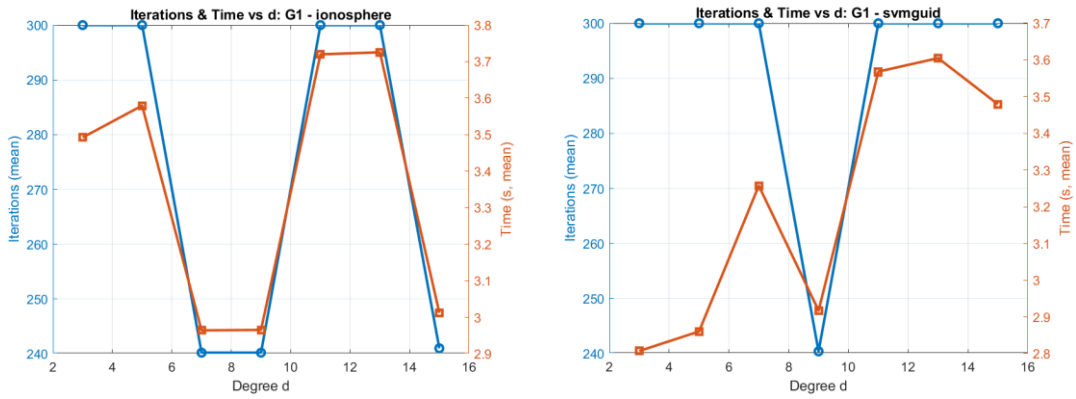
- When λ2 is larger, information spreads faster through the network, and consensus is achieved in fewer iterations.
- When λ2 is small, the consensus is slow, and the solver keeps iterating until it reaches the iteration cap.

3. The elapsed time of the solver in logarithmic scale



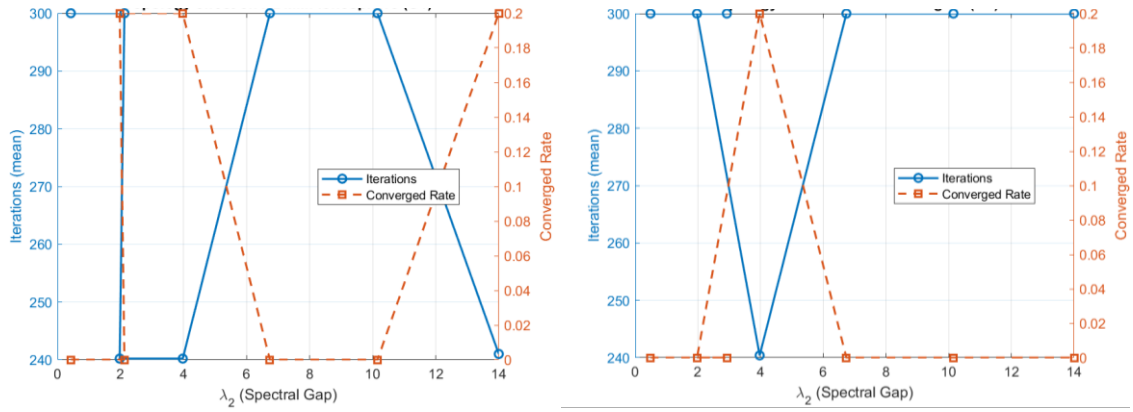
- **The elapsed time is shaped by a trade-off:** sparse graphs (low d) take long due to poor connectivity, while dense graphs (high d) incur higher communication cost. The fastest runtimes occur at **moderate degrees** (7–9), where spectral gap is large enough to speed convergence but communication overhead remains small.

4.iterations & time vs degree : ionosphere and svmguid (G1)



- Illustrating that solver performance is governed by the **spectral gap of the graph**: better connectivity reduces iterations and time, but because degree is only an indirect measure, the actual runtime shows fluctuations instead of a smooth trend.

5. Topology effect on ADMM: ionosphere and svmguid datasets (G1)



- This plot shows that ADMM's efficiency depends directly on the **spectral gap** of the graph. A larger λ2 means faster consensus (fewer iterations) and higher convergence rates, while a smaller λ2 leads to slow or failed convergence. Performance tracks λ2, not degree, which is why the curves are not monotonic.

G2 Datasets (Large Datasets)

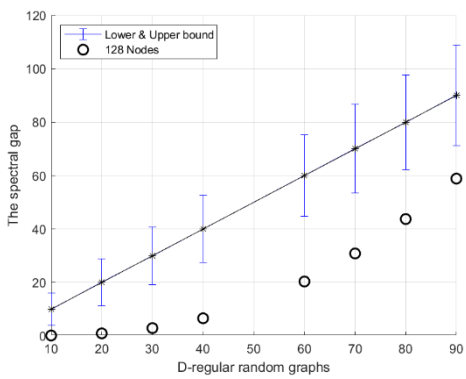
Datasets	Training Points	Testing Points	Features
phishing	11055	1655	68
a9a	32561	16281	123
ijcnn	35000	14990	22
skinNonskin	37492	5000	3

d-regular expander graphs with higher degrees designed for training these large datasets with 128 nodes

where d = {10,20,30,40,60,80,90}

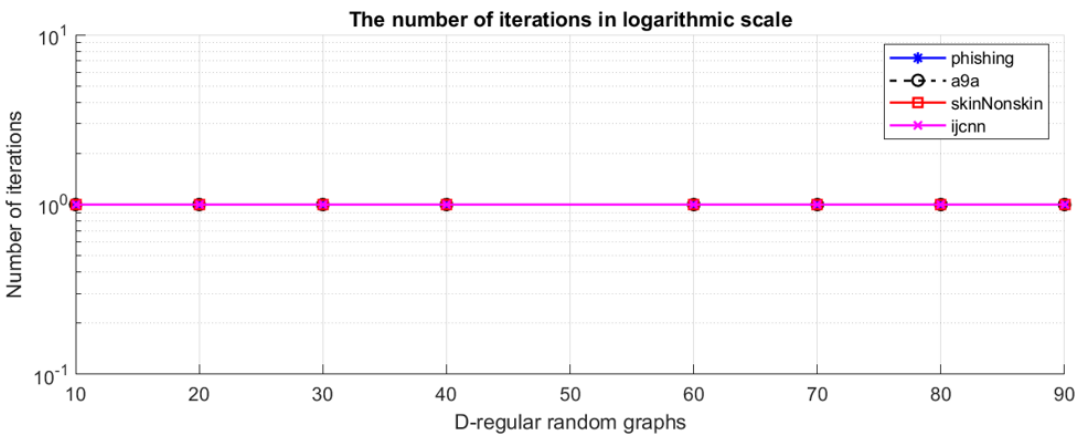
Results

1. The upper and lower bounds of the spectral gap for random d-regular expander graphs



This figure shows the upper and lower bounds of the spectral gap (λ2) for random d-regular and Ramanujan graphs with 128 nodes, where d = {10, 20, 30, 40, 60, 80, 90}.
The vertical bars represent the theoretical range of λ2 while the plotted points indicate the actual spectral gaps measured from the constructed graphs.
The Ramanujan graph values (marked with *) lie close to the upper bound, reflecting their near-optimal connectivity.
The random d-regular graph values (marked with o) fall within the theoretical bounds but may vary depending on the specific graph instance.
As the degree increases, the spectral gap generally becomes larger, indicating better connectivity and faster potential convergence in consensus algorithms.
Overall, the figure confirms that the graphs used in the experiments respect the theoretical spectral gap limits and validates their suitability for evaluating ADMM performance.

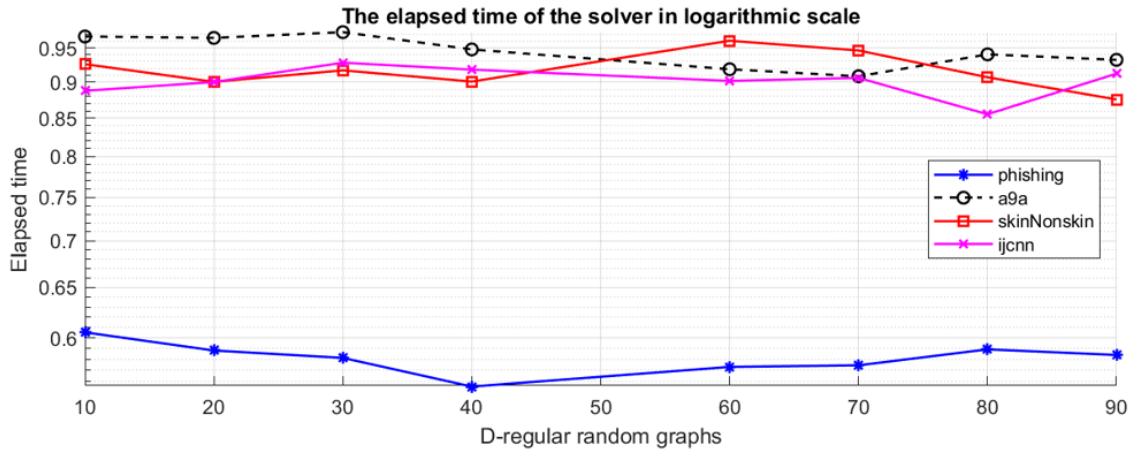
2. The number of iterations in logarithmic scale



This figure shows how the number of ADMM iterations varies with the degree of 128-node d-regular graphs for the G2 datasets (phishing, a9a, skinNonskin, and ijcnn).
Across all datasets and degrees, the number of iterations remains essentially constant at one, indicating that consensus is reached almost immediately.
This behaviour demonstrates that for large datasets, the solver is not sensitive to graph connectivity once a basic level of communication is present.
The spectral gap provided by even modest degrees is sufficient to ensure immediate convergence.

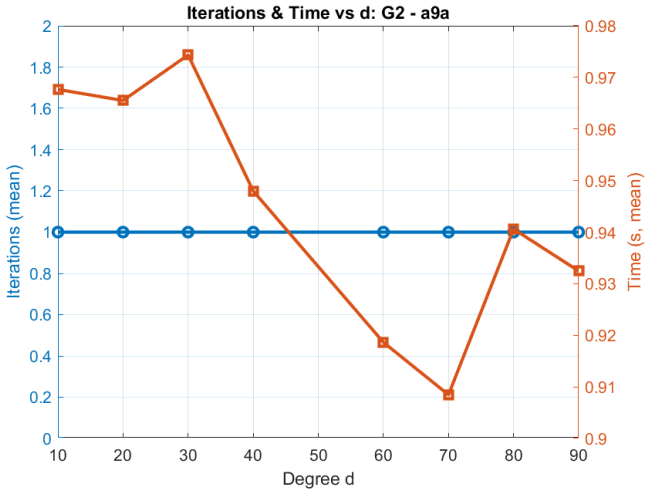
As a result, the iteration curves remain flat, and the graph highlights that topology has little to no effect on the iteration requirements for these datasets. This confirms that ADMM performance in G2 is dominated by dataset size and conditioning, not by the degree of the communication graph.

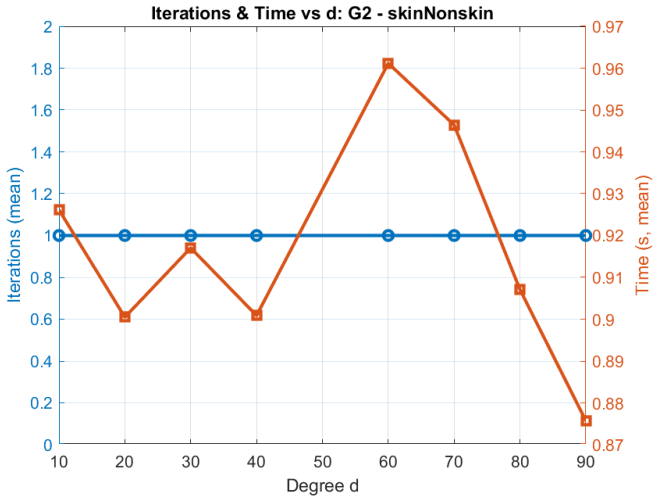
3. The elapsed time of the solver in logarithmic scale

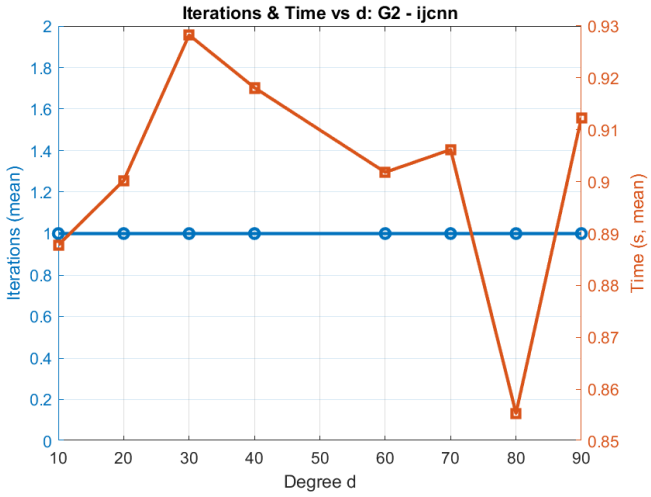


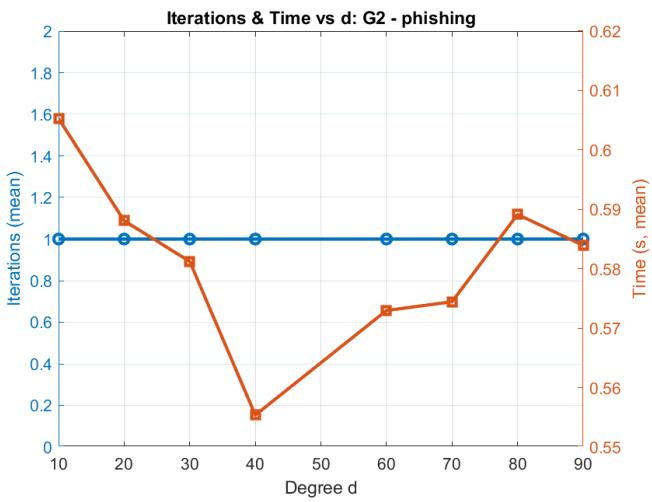
This figure illustrates the average elapsed runtime of the solver as the degree of 128-node d-regular graphs increases for the G2 datasets.
Since the number of iterations remains fixed at one, elapsed time does not vary significantly across degrees.
Small fluctuations that appear in the curves are caused by changes in communication overhead: as the degree grows, each iteration involves more neighbour exchanges, slightly increasing the per-iteration cost.
At moderate degrees, runtime tends to be marginally lower because communication remains efficient while connectivity is already sufficient.
At very high degrees, the cost per iteration rises slightly, leading to minor increases in runtime despite convergence being immediate.
Overall, this figure conveys that solver runtime for G2 datasets is nearly constant and that graph topology plays only a minimal role once datasets are large and well-conditioned.

4. Iterations & Time vs d: G2 Datasets



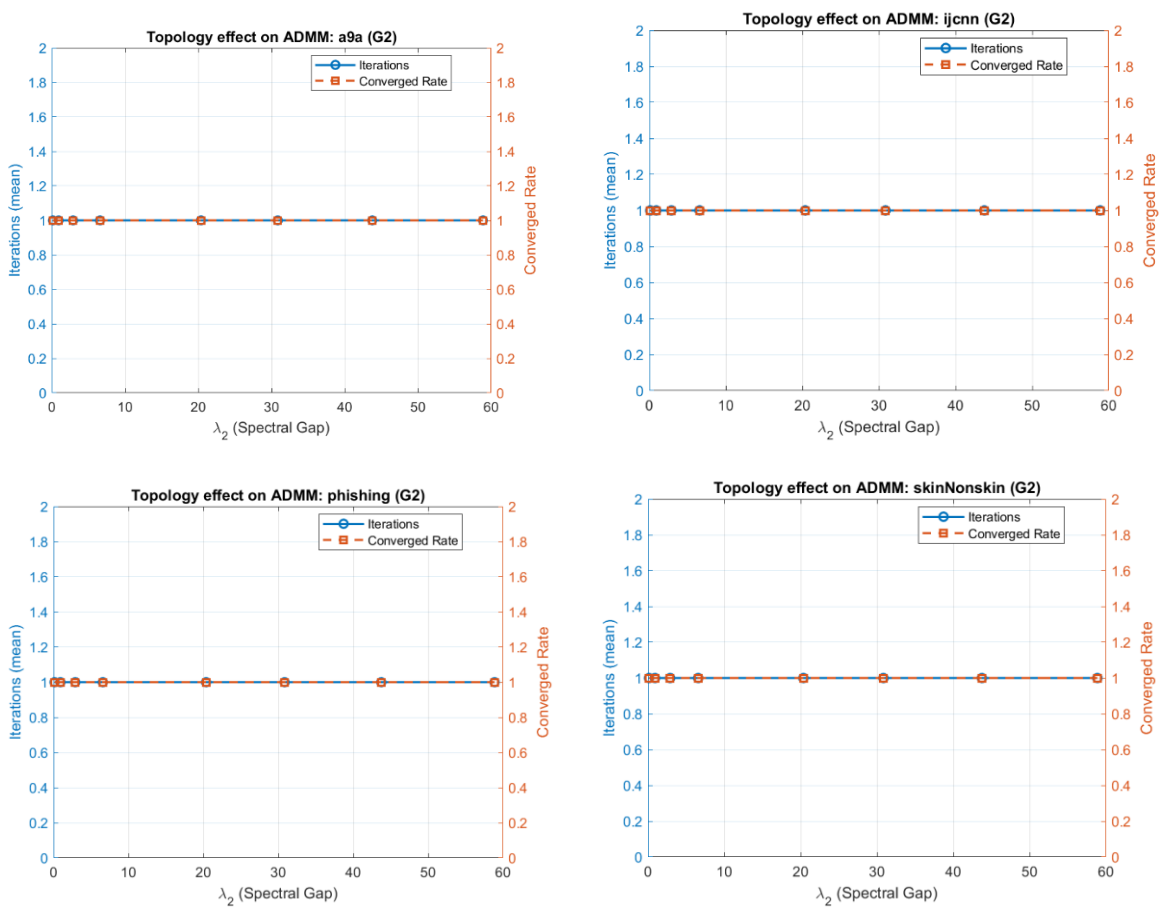






For large G2 datasets (a9a, ijcnn, phishing, skinNonskin), ADMM converges in a single step across all graph degrees. The runtime differences are marginal and only reflect per-iteration communication cost. This highlights that for big, well-conditioned problems, connectivity is sufficient even at low degrees, and topology has little effect on solver efficiency.

5. Topology effect on ADMM: G2 Datasets



For the G2 datasets, the topology effect plots indicate that the number of iterations remains constant at one across all spectral gap values.
The convergence rate stays at 100%, showing that the solver always reaches consensus successfully.
This demonstrates that spectral gap does not influence the convergence behaviour in these cases.
The large size of the datasets makes local models already well aligned before communication begins.
As a result, the ADMM algorithm converges in a single step, regardless of graph structure.
In contrast to G1 datasets, solver performance in G2 is independent of topology and consistently stable.

Analog Computing

Design and Implementation of an Analog Harmonic Oscillator Circuit

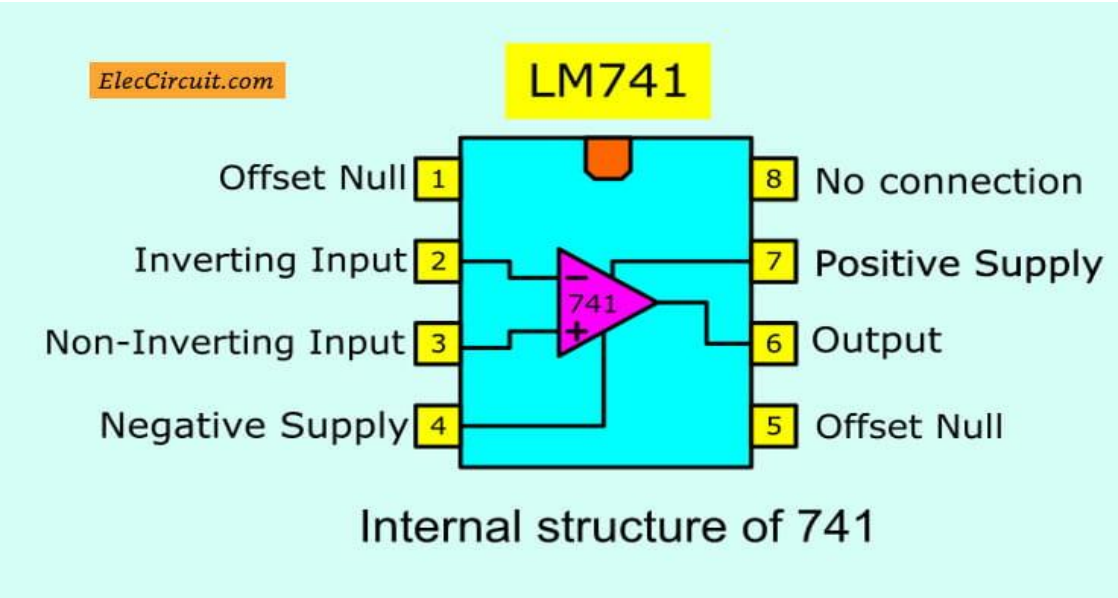
Introduction and Project Goals

Traditional computing relies on digital processors executing programmed instructions. This project explores an alternative computational paradigm: analog computing, where physical electronic components directly model and solve mathematical equations. We constructed a circuit-based analog computer capable of solving the differential equation $\mathbf{d^2x/dt^2 = -x}$, which mathematically describes Simple Harmonic Motion—the foundational behavior of oscillating systems found throughout physics and engineering.

Theoretical Framework

Understanding Our Primary Component: The 741 Op-Amp

Central to our design is the 741 operational amplifier integrated circuit. This component functions as a voltage-controlled amplifier with a critical characteristic: it automatically adjusts its output to equalize the voltage at two input terminals—Pin 2 (inverting) and Pin 3 (non-inverting). By grounding Pin 3 throughout our circuit, we create a condition where the op-amp constantly works to maintain Pin 2 at 0V (virtual ground). This simple rule drives all subsequent circuit behaviors.



Core Circuit Modules

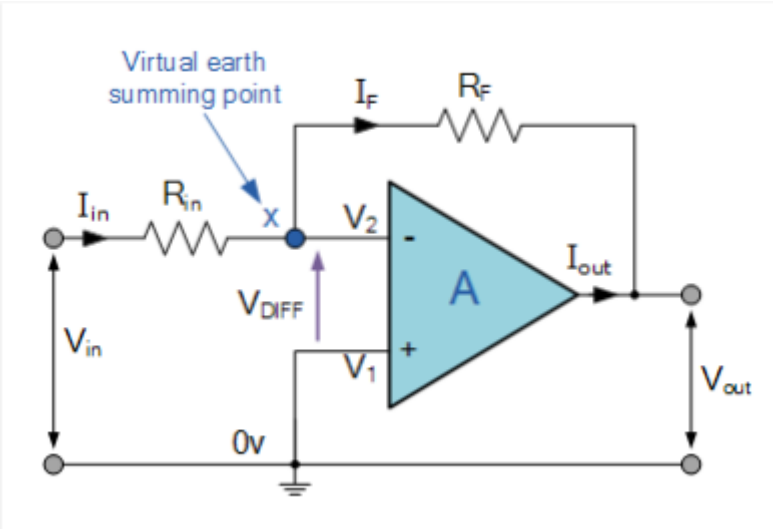
Our analog computer comprises two fundamental circuit types, each performing a specific mathematical operation:

Module Type A: Sign Inverter

Function: Produces an output voltage with inverted polarity relative to the input.

Construction: An op-amp configured with matched 10 kΩ resistors—the first resistor links the input signal to Pin 2, while the second creates a feedback connection from output (Pin 6) back to Pin 2.

Operational Theory: Input voltage generates current flow through the input resistor toward Pin 2. The op-amp responds by creating an output voltage that drives compensating current backward through the feedback resistor. This negative feedback mechanism inherently produces an inverted signal while maintaining Pin 2 at virtual ground potential.

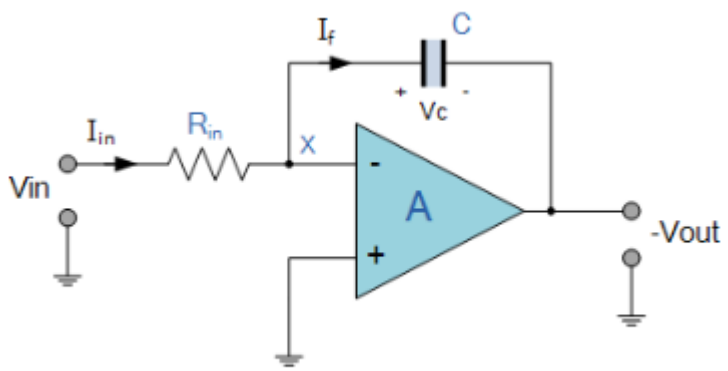


Module Type B: Time Integrator

Function: Generates an output representing the time-accumulated integral of the input signal.

Construction: An op-amp with a 10 kΩ input resistor and a 0.1 μF capacitor forming the feedback network between Pin 6 and Pin 2.

Operational Theory: Current from the input flows directly into charging the feedback capacitor, as the op-amp maintains Pin 2 at virtual ground. The capacitor's voltage increases proportionally to accumulated charge. Since the output must mirror this capacitor voltage, the result is a time-integration operation: output voltage equals the integral of input current over time.



System Architecture and Mathematical Solution

The Target Equation

Our circuit continuously solves: $\mathbf{d^2x/dt^2 = -x}$

This relationship states that acceleration equals negative displacement—the defining characteristic of harmonic oscillation. Physical examples include pendulums and spring-mass systems.

Three-Stage Feedback Architecture

We implement the solution through a cascaded feedback loop where each stage performs one mathematical operation:

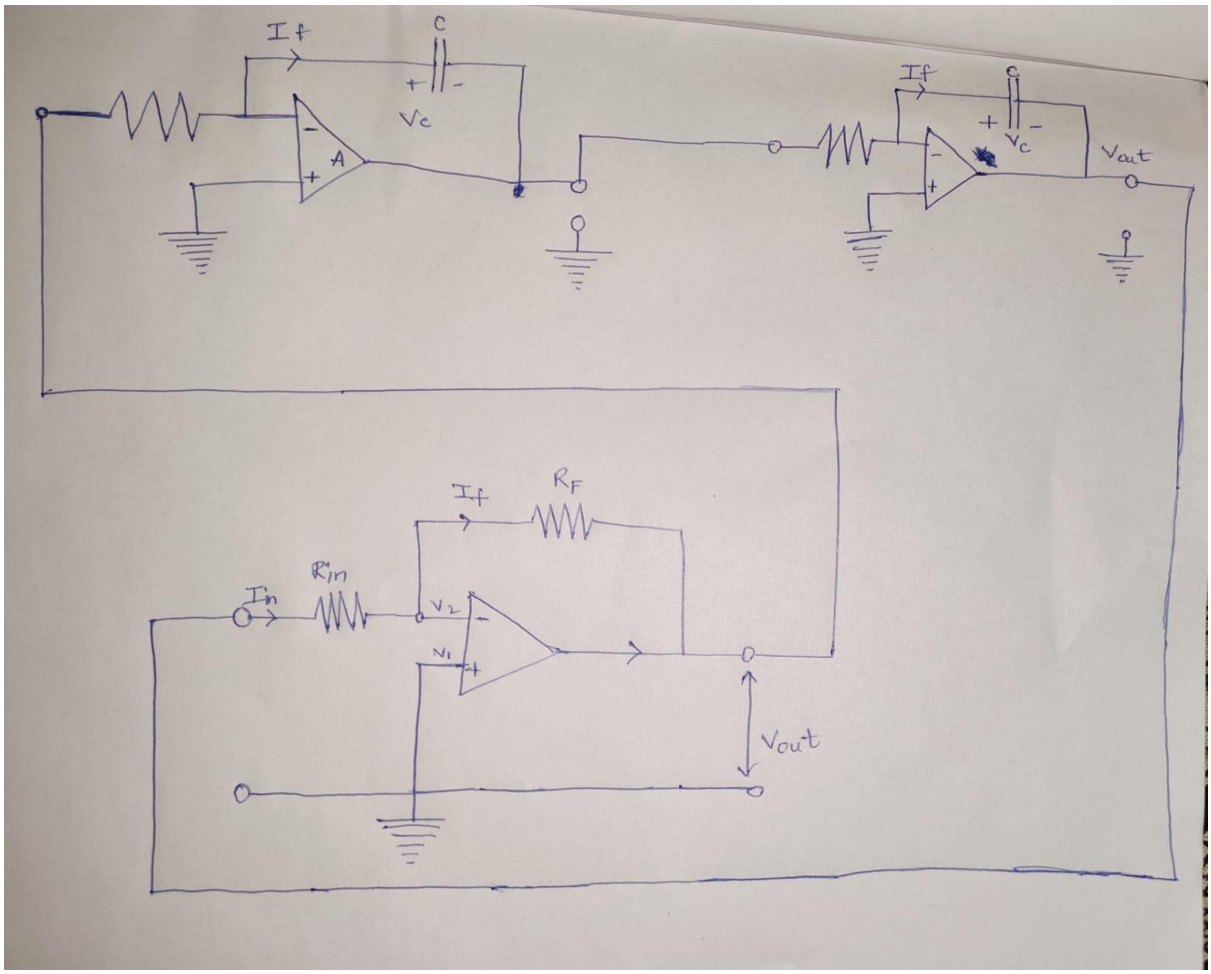
- **Third Stage Output** = Position (x)
- **Second Stage** = Integration block converting acceleration into velocity (-dx/dt)
- **First Stage** = Integration block converting velocity into position (x)

- **Loop Connection** = Inverter providing acceleration (-x) from position

This continuous loop maintains the mathematical relationship $d^2x/dt^2 = -x$ at every instant.

Required Components

Item Description	Count	Technical Specification
Operational Amplifier	3 units	741 IC, 8-pin configuration
Fixed Resistors	4 units	10 kΩ resistance value
Capacitors	2 units	0.1 μF capacitance
Prototyping Board	1	Standard breadboard
Connection Wires	Multiple	Assorted lengths
Test Equipment	1	ADALM2000 module



Construction Guidelines

Initial Setup: Power Distribution

Establish power connections from the ADALM2000: positive supply (V+) and negative supply (V-) connect to breadboard rails, with a separate ground reference rail. Each of the three 741 ICs requires power connections: Pin 7 connects to the positive rail, Pin 4 connects to the negative rail.

Building the Three Stages

First Stage - Sign Inverter (IC #1):
Ground connection to Pin 3. Input-to-Pin-2 via 10 kΩ resistor. Feedback from Pin 6 to Pin 2 via 10 kΩ resistor.

Second Stage - Primary Integrator (IC #2):
Ground connection to Pin 3. Input-to-Pin-2 via 10 kΩ resistor. Feedback from Pin 6 to Pin 2 via 0.1 μF capacitor.

Third Stage - Secondary Integrator (IC #3):
Ground connection to Pin 3. Input-to-Pin-2 via 10 kΩ resistor. Feedback from Pin 6 to Pin 2 via 0.1 μF capacitor.

Establishing the Computational Loop

Create continuous feedback by interconnecting stages: IC #3's output (Pin 6) feeds IC #1's input resistor. IC #1's output feeds IC #2's input resistor. IC #2's output feeds IC #3's input resistor, completing the loop.

Verification Procedure and Observed Results

Measurement Configuration

This circuit self-generates signals without requiring external stimulation. Configure the ADALM2000 oscilloscope with Channel 1 positive probe connected to IC #3 Pin 6 (observing position x), Channel 2 positive probe connected to IC #2 Pin 6 (observing velocity -dx/dt), and both ground probes connected to the circuit's ground reference.

Performance Indicators

Successful implementation produces two distinct waveforms: a smooth sinusoidal wave on Channel 1 representing position, and a cosinusoidal wave on Channel 2 representing velocity. These signals should maintain stable amplitude, consistent frequency, and a 90-degree phase offset. The presence of these continuous, stable oscillations confirms the circuit's ability to solve the differential equation in real-time through analog computation.

Summary

This project successfully demonstrates practical analog computing through physical circuit implementation. By cascading operational amplifier stages in a feedback configuration, we created a system that continuously performs mathematical operations equivalent to solving a second-order differential equation. The resulting stable harmonic oscillations validate the theoretical design and illustrate how electronic components can directly embody mathematical relationships, providing foundational understanding for advanced applications in signal processing and control systems.